Project Documentation: Round-Robin Load Balancing with Kubernetes, Docker and Linux

Submitted by: Pranav Mehta (Student, Chandigarh University, UID: 21BCS11480)

GitHub repository link: https://github.com/PranavMehta15/Kubernetes-Load-Balancing-Demo

Project Overview

This project involves setting up a round-robin load balancing system using Kubernetes. The system includes two servers (server1 and server2) and a client that interacts with these servers. The client performs requests to both servers, and Kubernetes manages the load balancing and fault tolerance.

Architecture

- **Servers**: Two server deployments (server1 and server2) that serve HTTP requests.
- **Client**: A client deployment that sends HTTP requests to both servers.
- **Load Balancing**: Managed by Kubernetes services that distribute traffic between server pods.
- **Fault Tolerance**: Handled by Kubernetes, which automatically restarts failed pods.

Setup and Environment

Virtual Machine:

- Platform: Ubuntu
- Version: Ubuntu 24.04 LTS
- Virtualization: VirtualBox

Methodology

1. Setting Up the Environment

Install MicroK8s

MicroK8s was installed as a lightweight Kubernetes distribution:

sudo snap install microk8s --classic

Verify installation:

microk8s kubectl version

Enable Necessary Add-ons

Enabled essential add-ons for Kubernetes:

microk8s enable dns

microk8s enable storage

Project Files Overview

The project consists of several files for defining Kubernetes resources, Docker configurations, and application logic. Below is a summary of the files and their purposes:

1. **Kubernetes Configuration Files:**
    - client-deployment.yaml
    - server1-deployment.yaml
    - server2-deployment.yaml
    - client-service.yaml
    - server1-service.yaml
    - server2-service.yaml
    - network-policy.yaml
2. **Docker Configuration Files:**
    - Dockerfile (Client)
    - Dockerfile (Server 1)
    - Dockerfile (Server 2)
3. **Application Files:**
    - client.py
    - server1.py
    - server2.py

Deploy the client:

microk8s kubectl apply -f client-deployment.yaml

4. Testing and Verification

Verify Pods and Services

Check the status of pods:

microk8s kubectl get pods

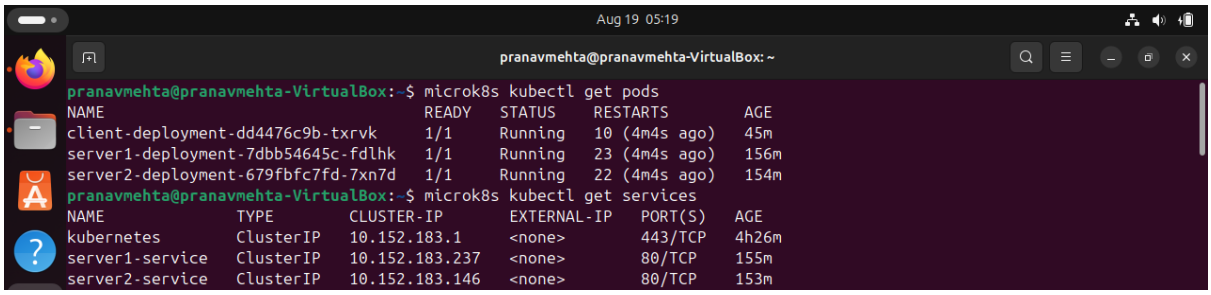

Image (i): Client server, Server1 & Server2 are up and running

Check the services:

microk8s kubectl get services

Simulate Failures

To demonstrate fault tolerance, delete a server pod:

microk8s kubectl delete pod <server1-pod-name>

Observe Kubernetes automatically restarting the pod.

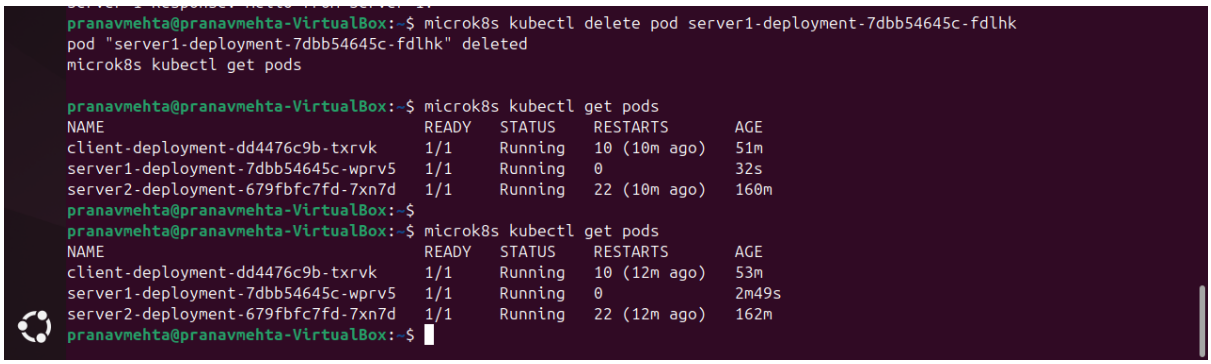

Image (ii): Deleting server1 to simulate a crash, resulting in Kubernetes automatically starting it again

Verify Load Balancing

Connect to the client pod and ensure it can communicate with both servers:

microk8s kubectl exec -it <client-pod-name> -- /bin/sh

curl http://server1-service

curl http://server2-service

Image (iii): Client server receives responses from both servers creating a loop as it follows round robin algorithm

Conclusion

The project demonstrates the use of Kubernetes for load balancing and fault tolerance in a microservices environment. The system effectively balances HTTP requests between multiple server instances and automatically handles server failures.

Commands Summary

- **Install MicroK8s**: sudo snap install microk8s --classic
- **Enable Add-ons**: microk8s enable dns, microk8s enable storage
- **Deploy Server 1**: microk8s kubectl apply -f server1-deployment.yaml
- **Create Server 1 Service**: microk8s kubectl apply -f server1-service.yaml
- **Deploy Client**: microk8s kubectl apply -f client-deployment.yaml
- **Check Pods**: microk8s kubectl get pods
- **Check Services**: microk8s kubectl get services
- **Simulate Failure**: microk8s kubectl delete pod <server1-pod-name>
- **Scale Deployments**: microk8s kubectl scale deployment/server1-deployment --replicas=3